

PORYGON: A Statistical and Rule-Based Runtime Security Framework for Containerized Workloads

23CSE498 – Project Phase 2 (Panel Review 1) | Team Number: 75

S.No	Register Number	Student Name
1	CB.EN.U4CSE23504	Ashin Varghese
2	CB.EN.U4CSE23559	Anurup Krishna
3	CB.EN.U4CSE23628	Krishnan S
4	CB.EN.U4CSE23661	Madhav Sreejith

Guide: **Dr Abirami K**, Assistant Professor, Department of CSE

September 6, 2026

Research Question and Contribution

The specific, falsifiable question this project answers

Container runtime anomaly detection (Falco, Sysdig, Aqua, and the academic line from Forrest et al. 1996 through Abed/Clancy 2015) all build a behavioural baseline of “normal” process activity and flag deviation. None of them publish a controlled measurement of **at what identity granularity that baseline should be scoped**: one profile per fleet (ARM-GLOBAL), per mutable tag (ARM-TAG), per exact immutable image digest (ARM-DIGEST), or per digest plus the container’s runtime security configuration (ARM-CONTEXT). Vendors and prior work pick one by engineering judgement.

This project’s contribution: a frozen, pre-registered protocol that runs identical Falco/eBPF telemetry and identical deterministic detection rules through all four scoping strategies on the same real Docker-container trials, and reports the false-positive-rate and recall outcome of each with exact statistical inference (McNemar’s test, Holm-Bonferroni correction, Clopper-Pearson intervals) – not a single reported accuracy figure.

Confirmed result (216 real Docker-container trials, 53 held-out benign runs, 96 planted-attack runs): scoping per image (ARM-CONTEXT) eliminates the false-positive rate entirely against a pooled baseline, 100% $\rightarrow$ 0% ($p = 6.7 \times 10^{-16}$), with zero loss of detection recall (96/96 both arms). This is a statistically decisive, reproduced result (first measured at $p = 5.8 \times 10^{-11}$ on 144 trials, confirmed at $p = 6.7 \times 10^{-16}$ on the full 216-trial run).

1. Methodology & Solution Design

Rubric: Methodology, Implementation Approach & Design Choices (5 Marks)

Methodological Framework:

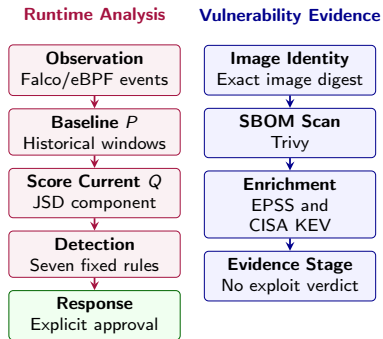
- **Behaviour Baseline:** Historical Docker lifecycle and Falco process events form a versioned baseline for one immutable image digest.
- **Anomaly Scoring:** For each categorical feature, the v1 scorer compares baseline P with a completed runtime window Q using base-2 Jensen–Shannon distance:

$$D_{JS}(P, Q) = \sqrt{JSD(P, Q)}$$

Smaller values indicate more similar behaviour.

- **Rule-Based Detection:** Seven deterministic rules produce explainable findings. An incident requires an unsuppressed, incident-eligible finding.

Porygon Workflow



1. Methodology & Solution Design (contd.)

Rubric: Methodology, Implementation Approach & Design Choices (5 Marks)

Implementation Approach and Design Choices:

- **Kernel-Level Monitoring:** Falco's modern eBPF driver captures container process execution without modifying the applications. The Telemetry Adapter ingests those events, while the Docker Collector records lifecycle and image-identity events. Both use bounded SQLite outboxes with retry delivery.
- **Service Integration:** Docker Compose coordinates eight long-running containers. Segregated networks restrict connectivity, and outbound access is limited to the vulnerability scanner.
- **Explainable Analysis:** Base-2 Jensen–Shannon distance provides a bounded, symmetric measure of categorical change. Fixed rules expose the matched evidence, so the implemented path does not depend on a black-box ML classifier.
- **Controlled Response:** Policy thresholds use severity and confidence to determine allowed containment actions. Disruptive execution is disabled by default. When enabled, it still requires explicit human approval.
- **Implementation Verification:** The repository contains 79 backend test functions across baseline, scoring, detection, calibration, response, and vulnerability modules, plus 131 further test functions across the collector, telemetry, responder, and scanner services (210 total).

2. Implementation Progress & Module Functionality

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

- **Implementation Status Overview:**

Module / Component	Current Implementation State	Status
Kernel Telemetry & Ingestion (Falco eBPF, Docker Collector, Telemetry Adapter)	Captures container process execution via eBPF; durable SQLite outbox with retry	Functional
Behavioural Baseline & JSD Scoring (baseline.py, scoring.py)	Digest-bound behavioural baselines; JSD-based anomaly distance scoring	Functional
Deterministic Detection Engine (detection.py)	7 correlation rules (POR-DET-001-007) generating findings/incidents	Functional
4-Arm Scope Comparison (experiments/)	216-trial confirmatory run analysed end to end, exact statistical pipeline	Complete

2. Implementation Progress (contd.)

Rubric: Core Modules Implemented, Functional & Integrated (8 Marks)

Module / Component	Current Implementation State	Status
Vulnerability & SBOM Pipeline (Scanner, Trivy 0.72.0)	CycloneDX SBOM generation; EPSS & CISA KEV enrichment	Functional
Response Policy Engine (response.py)	Severity/confidence-gated containment recommendations; human-approval required	Functional
Module Integration	8 containerized services integrated via Docker Compose with network segregation	Integrated

Core Functionality Demonstration: A real, functional XMRig cryptominer binary from a published malware-sample container image was executed against a live Porygon baseline built from 243 real Falco-captured events, with zero prior knowledge of the binary given to the system. Resulting anomaly score: 0.2696 (elevated band, threshold 0.25), with the executable and its parent-child call edge both flagged as previously unseen. Full record: docs/LIVE_DEMO_RECORD_V1.md.

3. Technical Knowledge & Design Decisions

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

- **Algorithmic Justification:** JSD is used over raw KL divergence because it is symmetric ($JSD(P, Q) = JSD(Q, P)$) and bounded, giving comparable scores across containers. Deterministic rules are used instead of a trained ML classifier because the system is explicitly designed around mathematical distance measures, not black-box classifiers, for explainability.

- **Mathematical Formulation:** The Response Policy Engine computes:

$$\text{Action} = \begin{cases} \text{stop} & \text{if } Sev \geq 0.90 \wedge Conf \geq 0.75 \wedge (\text{POR-DET-005} \vee \text{POR-DET-007}) \\ \text{pause} & \text{if } Sev \geq 0.70 \wedge Conf \geq 0.55 \\ \text{observe only} & \text{otherwise} \end{cases}$$

- **Confirmed Statistical Result:** the frozen protocol's primary hypothesis test (per-image scoping vs. pooled baseline) is significant at $n = 216$: exact McNemar $p = 6.7 \times 10^{-16}$ (Holm-adjusted), false-positive rate $100\% \rightarrow 0\%$, recall non-inferior (96/96 both arms).

3. Technical Knowledge & Design Decisions (contd.)

Rubric: Understanding of Methodology, Implementation & Design Choices (5 Marks)

- **Design Choices & Trade-offs:**

- **Multicall binary handling:** BusyBox/Toybox-style binaries are matched applet-aware, avoiding false positives from one executable exposing many command names.
- **Canonical scan identity:** Vulnerability scans are keyed by a SHA-256 hash of 6 parameters (image digest, local image ID, scanner name, scanner version, vulnerability schema version, researcher scan reference), making scans idempotent and immutable.
- **Network segregation:** Services run across three isolated Docker networks (ingress / internal / egress) with least-privilege socket access.
- **Fixed, versioned scoring weights:** every historical score stays reproducible under a named algorithm version; a weight change requires a new version, never a silent edit.

- **Confirmed Findings (216 real trials, exact statistical pipeline):**

- Per-image scoping vs. pooled baseline: significant, $p = 6.7 \times 10^{-16}$, FPR 100% $\rightarrow$ 0%, recall unchanged.
- Digest-plus-context vs. digest-only: no measurable difference in this dataset, $p = 1.0$, reproduced at both $n = 35$ and $n = 53$ – traced to the tested configuration change (a dropped Linux capability) not altering which processes execute, which the process-name feature cannot see by construction.

4. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

- **Individual MOOC / Online Certification Selection:**

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	Ashin Varghese	[Course Title relevant to Project]	Coursera	15 Hrs
2	Anurup Krishna	[Course Title relevant to Project]	NPTEL	20 Hrs
3	Krishnan S	[Course Title relevant to Project]	edX / Coursera	12 Hrs
4	Madhav Sreejith	[Course Title relevant to Project]	AWS / Google	16 Hrs

- **Requirement:** Each student must complete one approved 10–20 hour online certification from a recognized platform (Coursera, NPTEL, edX, etc.) relevant to the project.

References I



S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for Unix processes," in *Proc. IEEE Symp. Security and Privacy*, 1996, pp. 120–128.



S. A. Hofmeyr, S. Forrest, and A. Somayaji, "Intrusion detection using sequences of system calls," *Journal of Computer Security*, vol. 6, no. 3, pp. 151–180, 1998.



A. S. Abed, T. C. Clancy, and D. S. Levy, "Applying bag of system calls for anomalous behavior detection of applications in Linux containers," in *Proc. IEEE Globecom Workshops*, 2015, pp. 1–5.



J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.



A. Salem and F. Naït-Abdesselam, "Anomaly detection in network traffic using Jensen-Shannon divergence," in *Proc. IEEE International Conference on Communications (ICC)*, 2012.



X. Lin, Z. Chen, W. Yang, X. Yang, J. Luo, and X. Yang, "eBPF-Guard: a detection method for container escape via multi-level monitoring and enhanced analysis model," *Empirical Software Engineering*, vol. 31, article 51, 2026.



Li Wei, Yuan Zekun, Wu Kehe, and Cheng Rui, "Container anomaly detection based on attention mechanism and multiscale convolutional neural network," *Journal of Information Security Research*, vol. 11, no. 1, pp. 35–, 2025.



K. Xiong, Z. Wu, and X. Jia, "DeepContainer: a deep learning-based framework for real-time anomaly detection in cloud-native container environments," *Journal of Advanced Computing Systems*, 2025.

References II



J. H. Ring, C. M. Van Oort, S. Durst, V. White, J. P. Near, and C. Skalka, "Methods for host-based intrusion detection with deep learning," *Digital Threats: Research and Practice*, vol. 2, no. 4, 2021.



B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Boston, MA, USA: Addison-Wesley, 2019.



The Falco Authors, "Falco: cloud-native runtime security," Cloud Native Computing Foundation. [Online]. Available: <https://falco.org>



Open Container Initiative, "OCI image format specification." [Online]. Available: <https://github.com/opencontainers/image-spec>

Thank You